

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
по дисциплине «Мобильные и сетевые технологии»
НА ТЕМУ

Двоичные деревья поиска
15 Вариант

студент Северюхин Андрей Аркадьевич K3341
(Фамилия, Имя, Отчество) (группа)

Направление подготовки (специальность) 09.03.03
(код, наименование)

Мобильные и сетевые технологии

Квалификация Бакалавр
(бакалавр, магистр)*

Преподаватель Ромакина О.М.
(Фамилия, И. О.) (Подпись)

Дата выполнения: 21.09.2026

Санкт-Петербург
2026 г.

1 задача.

Обход двоичного дерева (1 балл)

В этой задаче вы реализуете три основных способа обхода двоичного дерева «в глубину»: центрированный (in-order), прямой (pre-order) и обратный (post-order). Очень полезно попрактиковаться в их реализации, чтобы лучше понять бинарные деревья поиска.

Вам дано корневое двоичное дерево. Выведите центрированный (in-order), прямой (pre-order) и обратный (post-order) обходы в глубину.

Формат ввода: стандартный ввод или input.txt. В первой строке входного файла содержится количество узлов n . Узлы дерева пронумерованы от 0 до $n - 1$. Узел 0 является корнем.

Следующие n строк содержат информацию об узлах 0, 1, ..., $n - 1$ по порядку. Каждая из этих строк содержит три целых числа K_i , L_i и R_i . K_i – ключ i -го узла, L_i – индекс левого ребенка i -го узла, а R_i – индекс правого ребенка i -го узла. Если у i -го узла нет левого или правого ребенка (или обоих), соответствующие числа L_i или R_i (или оба) будут равны -1 .

Ограничения на входные данные.

$1 \leq n \leq 10^5$, $0 \leq K_i \leq 10^9$, $-1 \leq L_i, R_i \leq n - 1$. Гарантируется, что данное дерево является двоичным деревом. В частности, если $L_i \neq -1$ и $R_i \neq -1$, то $L_i \neq R_i$. Кроме того, узел не может быть ребенком двух разных узлов. Кроме того, каждый узел является потомком корневого узла.

Формат вывода / выходного файла (output.txt). Выведите три строки. Первая строка должна содержать ключи узлов при центрированном обходе дерева (in-order). Вторая строка должна содержать ключи узлов при прямом обходе дерева (pre-order). Третья строка должна содержать ключи узлов при обратном обходе дерева (post-order).

Ограничение по времени. 5 сек.

Ограничение по памяти. 512 мб.

Что делать. Реализовать алгоритмы обхода из лекции. Обратите внимание, что в этой задаче дерево может быть очень глубоким, поэтому будьте осторожны, избегайте проблем с переполнением стека, если используете рекурсию, и обязательно протестируйте свое решение на дереве максимально возможной высоты.

Описание решения

Все три обхода это один и тот же спуск по дереву, отличается момент записи ключа (до детей, между ними, после)

Вместо реализации рекурсией (тк глубина может превысить допустимую) сделана итеративная версия через стек.

- Pre-order. достаем вершину из стека, пишем ключ, кладем обратно детей. Правого первым, чтобы левый достался раньше.
- Post-order. то же самое, но детей кладем наоборот. Получается корень, правый, левый - это развернутый post-order, поэтому в конце разворачиваем список.
- In-order. идем влево до конца, складывая вершины в стек. когда в конце - достаем вершину, пишем ключ, идем направо.

Асимптотика $O(n)$

Решение

```
def in_order(key, left, right):
    result, stack = [], []
    v = 0
    while stack or v != -1:
        while v != -1:
            stack.append(v)
            v = left[v]
        v = stack.pop()
        result.append(key[v])
        v = right[v]
    return result

def pre_order(key, left, right):
    result, stack = [], [0]
    while stack:
        v = stack.pop()
        result.append(key[v])
        if right[v] != -1:
            stack.append(right[v])
        if left[v] != -1:
            stack.append(left[v])
    return result

def post_order(key, left, right):
    result, stack = [], [0]
    while stack:
        v = stack.pop()
        result.append(key[v])
        if left[v] != -1:
            stack.append(left[v])
        if right[v] != -1:
            stack.append(right[v])
    result.reverse()
    return result

n = int(input())
key, left, right = [], [], []
for _ in range(n):
    k, l, r = map(int, input().split())
    key.append(k)
    left.append(l)
    right.append(r)

print(*in_order(key, left, right))
print(*pre_order(key, left, right))
print(*post_order(key, left, right))
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x16
ok 01_sample1          0.01 c    7.8 M6
ok 02_sample2          0.01 c    7.7 M6
ok 03_single           0.01 c    7.9 M6
ok 04_not_bst_duplicates 0.01 c    7.9 M6
ok 05_only_left_child  0.01 c    7.6 M6
ok 06_only_right_child 0.01 c    7.8 M6
ok 07_perfect_1023     0.01 c    8.1 M6
ok 12_key_bounds       0.01 c    7.6 M6
ok big_08_left_chain_1e5 0.16 c   20.0 M6
ok big_09_right_chain_1e5 0.17 c   19.3 M6
ok big_10_zigzag_1e5    0.17 c   19.2 M6
ok big_11_random_1e5    0.17 c   21.0 M6
OK 2.1 Обход двоичного дерева lab2/2.1.py 12 тестов, макс 0.17 с (лимит 5 с), 21 M6

Всего запусков: 12, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе решения данной задачи познакомились со структурой двоичного дерева, реализовали классические алгоритмы обхода, использовали итеративное решение через стек вместо рекурсивного решения.

5 задача.

Простое двоичное дерево поиска (1 балл)

Реализуйте простое двоичное дерево поиска.

Формат ввода / входного файла (input.txt). Входной файл содержит описание операций с деревом, их количество N не превышает 100. В каждой строке находится одна из следующих операций:

- insert x – добавить в дерево ключ x . Если ключ x есть в дереве, то ничего делать не надо;
- delete x – удалить из дерева ключ x . Если ключа x в дереве нет, то ничего делать не надо;
- exists x – если ключ x есть в дереве выведите «true», если нет – «false»;
- next x – выведите минимальный элемент в дереве, строго больший x , или «none», если такого нет;
- prev x – выведите максимальный элемент в дереве, строго меньший x , или «none», если такого нет.

В дерево помещаются и извлекаются только целые числа, не превышающие по модулю 10^9 .

Ограничения на входные данные.

$$0 \leq N \leq 100, |x_i| \leq 10^9.$$

Формат вывода / выходного файла (output.txt). Выведите последовательно результат выполнения всех операций exists, next, prev. Следуйте формату выходного файла из примера.

Ограничение по времени. 2 сек.

Ограничение по памяти. 512 мб.

Описание решения

BST без балансировки, операций всего 100 - глубина не важна.

Вставка и поиск очевидны. Идем от корня, меньше -> влево, больше -> вправо.

Удаление разбивается на три случая. Если у вершины нет одного из детей, подставляем на ее место второго. Если детей два, так сделать нельзя, поэтому берем минимум из правого поддерева, переносим его ключ в нашу вершину и удаляем уже его - у минимума точно нет левого ребенка, так что этот случай простой.

Next и prev - это спуск с запоминанием. Ищем минимальный ключ больше x : если текущая вершина больше x , запоминаем ее как кандидата и идем влево искать что-то поменьше, иначе идем вправо. Prev аналогично.

Асимптотика $O(N^2)$

Решение (1 часть)

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

```
def insert(node, x):
    if node is None:
        return Node(x)
    if x < node.key:
        node.left = insert(node.left, x)
    elif x > node.key:
        node.right = insert(node.right, x)
    return node
```

```
def delete(node, x):
    if node is None:
        return None
    if x < node.key:
        node.left = delete(node.left, x)
    elif x > node.key:
        node.right = delete(node.right, x)
    else:
        if node.left is None:
            return node.right
        if node.right is None:
            return node.left
        m = node.right
        while m.left is not None:
            m = m.left
        node.key = m.key
        node.right = delete(node.right, m.key)
    return node
```

```
def exists(node, x):
    while node is not None:
        if x == node.key:
            return True
        node = node.left if x < node.key else node.right
    return False
```

Решение (2 часть)

```
def next_key(node, x):
    res = None
    while node is not None:
        if node.key > x:
            res = node.key
            node = node.left
        else:
            node = node.right
    return res
```

```
def prev_key(node, x):
    res = None
    while node is not None:
        if node.key < x:
            res = node.key
            node = node.right
        else:
            node = node.left
    return res
```

```
import sys

root = None

for line in sys.stdin:
    parts = line.split()
    if not parts:
        continue
    op, x = parts[0], int(parts[1])
    if op == "insert":
        root = insert(root, x)
    elif op == "delete":
        root = delete(root, x)
    elif op == "exists":
        print("true" if exists(root, x) else "false")
    elif op == "next":
        r = next_key(root, x)
        print(r if r is not None else "none")
    elif op == "prev":
        r = prev_key(root, x)
        print(r if r is not None else "none")
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x15
ok 01_sample 0.01 c 7.9 M6
ok 02_empty_input 0.01 c 7.9 M6
ok 03_empty_tree_queries 0.01 c 7.9 M6
ok 04_duplicates 0.01 c 7.6 M6
ok 05_delete_root_two_children 0.01 c 7.9 M6
ok 06_delete_one_child 0.01 c 7.9 M6
ok 07_bounds 0.01 c 7.9 M6
ok 08_strict_next_prev 0.01 c 7.9 M6
ok 09_chain_then_delete 0.01 c 8.0 M6
ok 10_random_100_ops 0.01 c 7.9 M6
ok 11_negative_keys 0.01 c 7.9 M6
OK 2.5 Простое двоичное дерево поиска lab2/2.5.py 11 тестов, макс 0.01 с (лимит 2 с), 8 M6
Всего запусков: 11, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе решения данной задачи изучили и реализовали основные операции с BST.

10 задача.

Проверка корректности (2 балла)

Свойство двоичного дерева поиска можно сформулировать следующим образом: для каждой вершины дерева выполняется следующее условие:

- все ключи вершин из левого поддеревья меньше ключа вершины V ;
- все ключи вершин из правого поддеревья больше ключа вершины V .

Дано двоичное дерево. Проверьте, выполняется ли для него свойство двоичного дерева поиска.

Формат ввода / входного файла (input.txt). Входной файл содержит описание двоичного дерева.

В первой строке файла находится число N – число вершин в дереве. В последующих N строках файла находятся описания вершин дерева. В $(i + 1)$ -ой строке файла ($1 \leq i \leq N$) находится описание i -ой вершины, состоящее из трех чисел K_i , L_i , R_i , разделенных пробелами – ключа K_i в i -ой вершине, номера левого L_i ребенка i -ой вершины ($i < L_i \leq N$ или $L_i = 0$, если левого ребенка нет) и номера правого R_i ребенка i -ой вершины ($i < R_i \leq N$ или $R_i = 0$, если правого ребенка нет).

Ограничения на входные данные.

$0 \leq N \leq 2 \cdot 10^5$, $|K_i| \leq 10^9$.

На 60% от при $0 \leq N \leq 2000$.

Формат вывода / выходного файла (output.txt). Выведите «YES», если данное во входном файле дерево является двоичным деревом поиска, и «NO», если не является.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

Проверить можно по ссылке (<https://clck.ru/ebzED>), OpenEdu, курс "Алгоритмы программирования и структуры данных 6 неделя, наблюдаемая задача.

Описание решения

Нужно проверять вершины ниже текущего корня поддерева. Каждая из них обязана лежать в своем диапазоне.

Спускаемся по дереву и несем с собой две границы - минимум и максимум, в которые ключ обязан попасть. У корня границ нет поэтому ставим плюс и минус бесконечность. Когда уходим влево, максимумом становится ключ родителя. Когда вправо - минимумом.

Если ключ вершины в свои границы не попал, пишем NO.

Как и в прошлой задаче - реализация итеративная через стек.
Асимптотика $O(N)$

Решение

```
n = int(input())
key = [0] * (n + 1)
left = [0] * (n + 1)
right = [0] * (n + 1)

for i in range(1, n + 1):
    k, l, r = map(int, input().split())
    key[i] = k
    left[i] = l
    right[i] = r

inf = float('inf')
ok = True
stack = [(1, -inf, inf)] if n > 0 else []

while stack:
    v, lo, hi = stack.pop()
    if not lo < key[v] < hi:
        ok = False
        break
    if left[v] != 0:
        stack.append((left[v], lo, key[v]))
    if right[v] != 0:
        stack.append((right[v], key[v], hi))

if ok:
    print("YES")
else:
    print("NO")
```

Описание проведенных тестов

В связи с невозможностью просмотреть указанный курс на OpenEdu по техническим причинам, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x18
ok 01_sample_yes          0.01 c    7.7 M6
ok 02_sample_empty        0.01 c    7.8 M6
ok 03_sample_no           0.01 c    7.8 M6
ok 04_single              0.01 c    7.8 M6
ok 05_equal_keys          0.01 c    7.8 M6
ok 06_equal_right         0.01 c    7.8 M6
ok 07_grandchild          0.01 c    7.8 M6
ok 08_n2000_yes           0.01 c    7.9 M6
ok 14_all_equal           0.01 c    7.9 M6
ok big_09_chain_2e5_yes   0.23 c   24.7 M6
ok big_10_left_chain_2e5_yes 0.23 c   24.7 M6
ok big_11_random_2e5_yes  0.24 c   24.8 M6
ok big_12_deep_violation  0.22 c   24.9 M6
ok big_13_chain_last_bad  0.23 c   24.7 M6
OK 2.10 Проверка корректности lab2/2.10.py 14 тестов, макс 0.24 с (лимит 2 с), 25 M6

Всего запусков: 14, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В результате задача решена, применен итеративный подход вместо рекурсивного. Также уяснил, что свойство BST не локальное. Проверки родитель больше левого ребенка недостаточно потому что условие накладывается на все поддереву целиком, а не только на соседей.

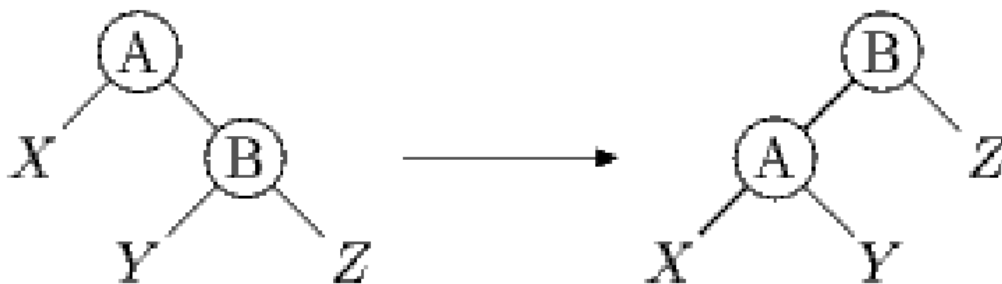
13 задача.

Делаю я левый поворот... (3 балла)

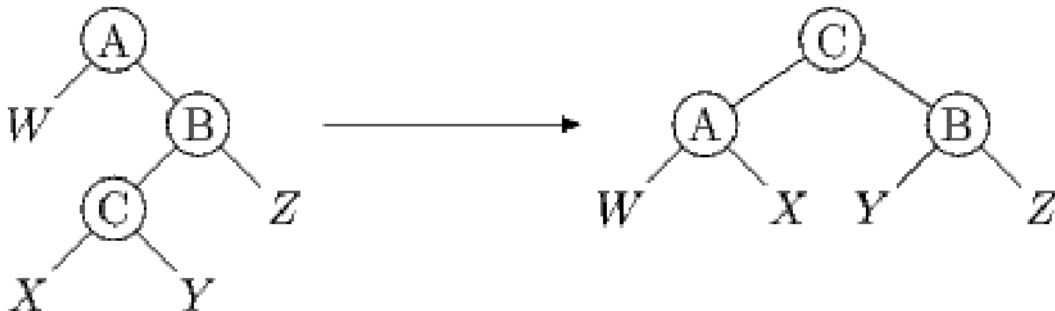
Для балансировки AVL-дерева при операциях вставки и удаления производятся левые и правые повороты. Левый поворот в вершине производится, когда баланс этой вершины больше 1, аналогично, правый поворот производится при балансе, меньшем -1.

Существует два разных левых (как, разумеется, и правых) поворота: большой и малый левый поворот.

Малый левый поворот осуществляется следующим образом:



Заметим, что если до выполнения малого левого поворота был нарушен баланс только корня дерева, то после его выполнения все вершины становятся сбалансированными, за исключением случая, когда у правого ребенка корня баланс до поворота равен -1. В этом случае вместо малого левого поворота выполняется большой левый поворот, который осуществляется так:



Дано дерево, в котором баланс корня равен 2. Сделайте левый поворот.

Формат ввода / входного файла (input.txt). Входной файл содержит описание двоичного дерева.

В первой строке файла находится число N – число вершин в дереве. В последующих N строках файла находятся описания вершин дерева. В $(i + 1)$ -ой строке файла $(1 \leq i \leq N)$ находится описание i -ой вершины, состоящее из трех чисел K_i, L_i, R_i , разделенных пробелами – ключа K_i в i -ой вершине, номера левого L_i ребенка i -ой вершины ($i < L_i \leq N$ или $L_i = 0$, если левого ребенка нет) и номера правого R_i ребенка i -ой вершины ($i < R_i \leq N$ или $R_i = 0$, если правого ребенка нет). Все ключи различны. Гарантируется, что данное дерево является деревом поиска. Баланс корня дерева (вершины с номером 1) равен 2, баланс всех остальных вершин находится в пределах от -1 до 1.

Ограничения на входные данные.

$3 \leq N \leq 2 \cdot 10^5$, $|K_i| \leq 10^9$.

Формат вывода / выходного файла (output.txt). Выведите в том же формате дерево после осуществления левого поворота. Нумерация вершин может быть произвольной при условии соблюдения формата. Так, номер вершины должен быть меньше номера ее детей.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

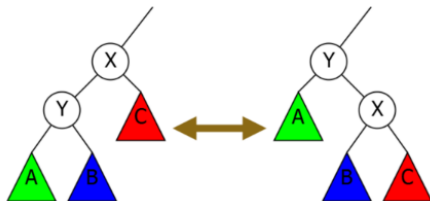
Проверить можно по ссылке (<https://clck.ru/ebzFQ>), OpenEdu, курс "Алгоритмы программирования и структуры данных 7 неделя, 2 задача.

Описание решения

Решение взято из материалов лекций:

Балансировка: правый поворот

- **Идея:** Нужно сделать перестановки в дереве, чтобы сохранить баланс
- **Проблема:** Как сохранить порядок при выполнении перестановок?
- Правый поворот:



$$A < Y < B < X < C$$

```
1 RotateRight(X):
2   P = X.Parent
3   Y = X.Left
4   B = Y.Right
5   Y.Parent = P
6   P.AppropriateChild = Y
7   X.Parent = Y
8   Y.Right = X
9   B.Parent = X
10  X.Left = B
```

Рассмотрим подробнее.

Поворот - это подъем ребенка на место родителя с переносом среднего поддеревя. Порядок при этом не ломается тк среднее поддерево по значениям лежит между двумя вершинами, которые меняются местами, и подходит обеим.

Задача в том, какой поворот выбрать. Если баланс правого ребенка корня равен -1, делаем большой, иначе малый. Значит нужны высоты.

Высоты можно посчитать без рекурсии - по формату номер вершины меньше номеров ее детей, поэтому идем с конца, дети уже посчитаны.

После поворота номера перепутались, родитель может оказаться ниже ребенка. Поэтому нумеруем заново обходом в ширину от нового корня

Асимптотика $O(N)$

Решение

```
n = int(input())
key, left, right = [0] * (n + 1), [0] * (n + 1), [0] * (n + 1)
for i in range(1, n + 1):
    k, l, r = map(int, input().split())
    key[i] = k
    left[i] = l
    right[i] = r

height = [0] * (n + 1)
for i in range(n, 0, -1):
    height[i] = 1 + max(height[left[i]], height[right[i]])

a = 1
b = right[a]
if height[right[b]] - height[left[b]] == -1:
    c = left[b]
    right[a] = left[c]
    left[b] = right[c]
    left[c] = a
    right[c] = b
    root = c
else:
    right[a] = left[b]
    left[b] = a
    root = b

new = [0] * (n + 1)
order = [root]
new[root] = 1
cnt = 1
i = 0
while i < len(order):
    v = order[i]
    i += 1
    if left[v] != 0:
        cnt += 1
        new[left[v]] = cnt
        order.append(left[v])
    if right[v] != 0:
        cnt += 1
        new[right[v]] = cnt
        order.append(right[v])

print(n)
for v in order:
    print(key[v], new[left[v]], new[right[v]])
```

Описание проведенных тестов

В связи с невозможностью просмотреть указанный курс на OpenEdu по техническим причинам, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x20
ok 01_sample 0.01 c 7.8 Mб
ok 02_min_small_rotation 0.01 c 7.8 Mб
ok 03_min_big_rotation 0.01 c 7.8 Mб
ok 04_small_rot_balance0 0.01 c 7.8 Mб
ok 05_small_rot_balance_plus 0.01 c 7.5 Mб
ok 06_big_rot 0.01 c 7.8 Mб
ok 07_big_rot_deeper 0.01 c 7.5 Mб
ok 08_negative_keys 0.01 c 7.8 Mб
ok 09_random_avl_h3 0.01 c 7.8 Mб
ok 10_random_avl_h4 0.01 c 7.8 Mб
ok 11_random_avl_h5 0.01 c 7.6 Mб
ok 12_random_avl_h6 0.01 c 7.8 Mб
ok big_13_small_rot_1e5 0.29 c 30.8 Mб
ok big_14_big_rot_1e5 0.23 c 26.3 Mб
ok big_15_chain_like 0.12 c 17.3 Mб
ok big_16_random_avl 0.05 c 11.1 Mб
OK 2.13 Делаю я левый поворот... lab2/2.13.py 16 тестов, макс 0.29 с (лимит 2 с), 31 Мб
Всего запусков: 16, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе решения данного разобрались поворотами в BST.

Дополнительные задачи

6 задача.

Опознавание двоичного дерева поиска (1.5 балла)

В этой задаче вы собираетесь проверить, правильно ли реализована структура данных бинарного дерева поиска. Другими словами, вы хотите убедиться, что вы можете находить целые числа в этом двоичном дереве, используя бинарный поиск по дереву, и вы всегда получите правильный результат: если целое число есть в дереве, вы его найдете, иначе – нет.

Вам дано двоичное дерево с ключами - целыми числами. Вам нужно проверить, является ли это правильным двоичным деревом поиска. Для каждой вершины дерева V выполняется следующее условие:

- все ключи вершин из левого поддерева меньше ключа вершины V ;
- все ключи вершин из правого поддерева больше ключа вершины V .

Другими словами, узлы с меньшими ключами находятся слева, а узлы с большими ключами – справа. Вам необходимо проверить, удовлетворяет ли данная структура двоичного дерева этому условию. Вам гарантируется, что входные данные содержат допустимое двоичное дерево. То есть это дерево, и каждый узел имеет не более двух ребенков.

Формат ввода / входного файла (input.txt). В первой строке входного файла содержится количество узлов n . Узлы дерева пронумерованы от 0 до $n - 1$. Узел 0 является корнем.

Следующие n строк содержат информацию об узлах 0, 1, ..., $n - 1$ по порядку. Каждая из этих строк содержит три целых числа K_i , L_i и R_i . K_i – ключ i -го узла, L_i - индекс левого ребенка i -го узла, а R_i - индекс правого ребенка i -го узла. Если у i -го узла нет левого или правого ребенка (или обоих), соответствующие числа L_i или R_i (или оба) будут равны -1 .

Ограничения на входные данные.

$0 \leq n \leq 10^5$, $-2^{31} \leq K_i \leq 2^{31} - 1$, $-1 \leq L_i, R_i \leq n - 1$. Гарантируется, что данное дерево является двоичным деревом. В частности, если $L_i \neq -1$ и $R_i \neq -1$, то $L_i \neq R_i$. Кроме того, узел не может быть ребенком двух разных узлов. Кроме того, каждый узел является потомком корневого узла. Все ключи во входных данных различны.

Формат вывода / выходного файла (output.txt). Если заданное двоичное дерево является правильным двоичным деревом поиска, выведите одно слово «CORRECT» (без кавычек). В противном случае выведите одно слово «INCORRECT» (без кавычек).

Ограничение по времени. 10 сек.

Ограничение по памяти. 512 мб.

Примечание. Пустое дерево считается правильным двоичным деревом поиска. Дерево не обязательно должно быть сбалансировано.

Описание решения

Сравнивать вершину только с детьми недостаточно.

Поэтому спускаемся и несем с собой границы, в которые ключ обязан попасть. У корня это плюс и минус бесконечность. Уходим влево - максимумом становится ключ родителя, вправо - минимумом. Если не попали в границы - пишем INCORRECT.

Асимптотика $O(n)$

Решение

```
n = int(input())
key = [0] * n
left = [0] * n
right = [0] * n

for i in range(n):
    k, l, r = map(int, input().split())
    key[i] = k
    left[i] = l
    right[i] = r

inf = float('inf')
ok = True
stack = [(0, -inf, inf)] if n > 0 else []

while stack:
    v, lo, hi = stack.pop()
    if not lo < key[v] < hi:
        ok = False
        break
    if left[v] != -1:
        stack.append((left[v], lo, key[v]))
    if right[v] != -1:
        stack.append((right[v], key[v], hi))

if ok:
    print("CORRECT")
else:
    print("INCORRECT")
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x17
ok 01_sample_correct      0.01 c    7.7 Mб
ok 02_sample_incorrect    0.01 c    7.8 Mб
ok 03_sample_empty        0.01 c    7.8 Mб
ok 04_sample_right_chain  0.01 c    7.8 Mб
ok 05_sample_perfect      0.01 c    7.8 Mб
ok 06_sample_grandchild   0.01 c    7.8 Mб
ok 07_single              0.01 c    7.8 Mб
ok 08_int32_bounds        0.01 c    7.8 Mб
ok 09_int32_bounds_bad    0.01 c    7.8 Mб
ok big_10_left_chain_1e5  0.12 c   16.2 Mб
ok big_11_random_1e5      0.13 c   16.2 Mб
ok big_12_deep_violation  0.12 c   16.2 Mб
ok big_13_chain_last_bad  0.12 c   16.0 Mб
OK 2.6 Оpozнание BST      lab2/2.6.py 13 тестов, макс 0.13 с (лимит 10 с), 16 Mб
Всего запусков: 13, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

Задача была решена.

7 задача.

Опознавание двоичного дерева поиска (усложненная версия) (2.5 балла)

Эта задача отличается от предыдущей тем, что двоичное дерево поиска может содержать равные ключи.

Вам дано двоичное дерево с ключами - целыми числами, которые могут повторяться. Вам нужно проверить, является ли это правильным двоичным деревом поиска. Теперь, для каждой вершины дерева V выполняется следующее условие:

- все ключи вершин из левого поддеревья меньше ключа вершины V ;
- все ключи вершин из правого поддеревья больше или равны ключу вершины V .

Другими словами, узлы с меньшими ключами находятся слева, а узлы с большими ключами – справа, дубликаты всегда справа. Вам необходимо проверить, удовлетворяет ли данная структура двоичного дерева этому условию.

Формат ввода / входного файла (input.txt). В первой строке входного файла содержится количество узлов n . Узлы дерева пронумерованы от 0 до $n - 1$. Узел 0 является корнем.

Следующие n строк содержат информацию об узлах 0, 1, ..., $n - 1$ по порядку. Каждая из этих строк содержит три целых числа K_i , L_i и R_i . K_i – ключ i -го узла, L_i - индекс левого ребенка i -го узла, а R_i - индекс правого ребенка i -го узла. Если у i -го узла нет левого или правого ребенка (или обоих), соответствующие числа L_i или R_i (или оба) будут равны -1 .

Ограничения на входные данные.

$0 \leq n \leq 10^5$, $-2^{31} \leq K_i \leq 2^{31} - 1$, $-1 \leq L_i, R_i \leq n - 1$. Гарантируется, что данное дерево является двоичным деревом. В частности, если $L_i \neq -1$ и $R_i \neq -1$, то $L_i \neq R_i$. Кроме того, узел не может быть ребенком двух разных узлов. Кроме того, каждый узел является потомком корневого узла. Обратите внимание, что минимальное и максимальное возможные значения 32-битного целочисленного типа могут быть ключами в дереве.

Формат вывода / выходного файла (output.txt). Если заданное двоичное дерево является правильным двоичным деревом поиска, выведите одно слово «CORRECT» (без кавычек). В противном случае выведите одно слово «INCORRECT» (без кавычек).

Ограничение по времени. 10 сек.

Ограничение по памяти. 512 мб.

Примечание. Пустое дерево считается правильным двоичным деревом поиска. Дерево не обязательно должно быть сбалансировано. Попробуйте адаптировать алгоритм из предыдущей задачи к случаю, когда допускаются повторяющиеся ключи, и остерегайтесь целочисленного переполнения!

Описание решения

Все то же самое, что и в прошлой задаче, меняется только одно неравенство.

Теперь дубликаты разрешены, но только справа. Значит в правом поддереве ключ может равняться ключу родителя, а в левом нет. Нижняя граница становится нестрогой, верхняя остается строгой

Асимптотика $O(n)$

Решение

```
n = int(input())
key = [0] * n
left = [0] * n
right = [0] * n

for i in range(n):
    k, l, r = map(int, input().split())
    key[i] = k
    left[i] = l
    right[i] = r

inf = float('inf')
ok = True
stack = [(0, -inf, inf)] if n > 0 else []

while stack:
    v, lo, hi = stack.pop()
    if not lo <= key[v] < hi:
        ok = False
        break
    if left[v] != -1:
        stack.append((left[v], lo, key[v]))
    if right[v] != -1:
        stack.append((right[v], key[v], hi))

if ok:
    print("CORRECT")
else:
    print("INCORRECT")
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x22
ok 01_sample_correct      0.01 c    7.8 M6
ok 02_sample_incorrect    0.01 c    7.5 M6
ok 03_sample_dup_right    0.01 c    7.8 M6
ok 04_sample_dup_left     0.01 c    7.8 M6
ok 05_sample_right_chain  0.01 c    7.8 M6
ok 06_sample_perfect      0.01 c    7.8 M6
ok 07_sample_intmax       0.01 c    7.5 M6
ok 08_empty               0.01 c    7.8 M6
ok 09_all_equal_right_chain 0.01 c    7.9 M6
ok 10_all_equal_left_chain 0.01 c    7.8 M6
ok 11_dup_deeper_right    0.01 c    7.8 M6
ok 12_dup_hidden_left     0.01 c    7.8 M6
ok 13_int32_dup           0.01 c    7.8 M6
ok 14_int32_dup_right     0.01 c    7.8 M6
ok 15_int32_span          0.01 c    7.8 M6
ok big_16_many_duplicates 0.12 c   15.8 M6
ok big_17_deep_violation  0.12 c   16.2 M6
ok big_18_random_correct  0.13 c   16.2 M6
OK 2.7 Опознание BST (усложненная) lab2/2.7.py 18 тестов, макс 0.13 с (лимит 10 с), 16 M6

Всего запусков: 18, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе решения данной задачи выяснили, что подход с границами легко переносится на другие условия. Правило дубликаты только справа сводится к замене одного знака:
- нижняя граница стала нестрогой, а верхняя осталась строгой.

12 задача.

Проверка сбалансированности (2 балла)

АВЛ-дерево является сбалансированным в следующем смысле: для любой вершины высота ее левого поддерева отличается от высоты ее правого поддерева не больше, чем на единицу.

Введем понятие баланса вершины: для вершины дерева V ее баланс $B(V)$ равен разности высоты правого поддерева и высоты левого поддерева. Таким образом, свойство АВЛ-дерева, приведенное выше, можно сформулировать следующим образом: для любой ее вершины V выполняется следующее неравенство:

$$-1 \leq B(V) \leq 1$$

Обратите внимание, что, по историческим причинам, определение баланса в этой и последующих задачах этой недели «зеркально отражено» по сравнению с определением баланса в лекциях! Надеемся, что этот факт не доставит Вам неудобств. В литературе по алгоритмам – как российской, так и мировой – ситуация, как правило, примерно та же.

Дано двоичное дерево поиска. Для каждой его вершины требуется определить ее баланс.

Формат ввода / входного файла (input.txt). Входной файл содержит описание двоичного дерева.

В первой строке файла находится число N – число вершин в дереве. В последующих N строках файла находятся описания вершин дерева. В $(i + 1)$ -ой строке файла ($1 \leq i \leq N$) находится описание i -ой вершины, состоящее из трех чисел K_i , L_i , R_i , разделенных пробелами – ключа K_i в i -ой вершине, номера левого L_i ребенка i -ой вершины ($i < L_i \leq N$ или $L_i = 0$, если левого ребенка нет) и номера правого R_i ребенка i -ой вершины ($i < R_i \leq N$ или $R_i = 0$, если правого ребенка нет). Все ключи различны. Гарантируется, что данное дерево является деревом поиска.

Ограничения на входные данные.

$$0 \leq N \leq 2 \cdot 10^5, |K_i| \leq 10^9.$$

Формат вывода / выходного файла (output.txt). Для i -ой вершины в i -ой строке выведите одно число – баланс данной вершины.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

Проверить можно по ссылке (<https://clck.ru/dZTVm>), OpenEdu, курс "Алгоритмы программирования и структуры данных 7 неделя, 1 задача.

Описание решения

Баланс вершины это разность высот ее поддеревьев, значит вся задача сводится к тому, чтобы посчитать высоты. Рекурсией делать нельзя тк дерево может быть цепочкой из 200000 вершин. Но по формату номер вершины меньше номеров ее детей, поэтому идем с конца и дети всегда уже посчитаны (т.е. рекурсия не нужна)
Асимптотика $O(N)$

Решение

```
n = int(input())
key = [0] * (n + 1)
left = [0] * (n + 1)
right = [0] * (n + 1)

for i in range(1, n + 1):
    k, l, r = map(int, input().split())
    key[i] = k
    left[i] = l
    right[i] = r

height = [0] * (n + 1)

for i in range(n, 0, -1):
    height[i] = 1 + max(height[left[i]], height[right[i]])

for i in range(1, n + 1):
    print(height[right[i]] - height[left[i]])
```

Описание проведенных тестов

В связи с невозможностью просмотреть указанный курс на OpenEdu по техническим причинам, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x16
ok 01_sample          0.01 c    7.6 Mб
ok 02_empty           0.01 c    7.7 Mб
ok 03_single           0.01 c    7.5 Mб
ok 04_right_chain_10   0.01 c    7.5 Mб
ok 05_left_chain_10    0.01 c    7.7 Mб
ok 06_perfect_31       0.01 c    7.7 Mб
ok 07_random_1000      0.01 c    7.9 Mб
ok 08_dfs_numbering    0.01 c    7.8 Mб
ok 12_key_bounds       0.01 c    7.8 Mб
ok big_09_chain_2e5    0.26 c   32.6 Mб
ok big_10_zigzag_2e5   0.27 c   32.6 Mб
ok big_11_random_2e5   0.27 c   26.6 Mб
OK 2.12 Проверка сбалансированности lab2/2.12.py 12 тестов, макс 0.27 с (лимит 2 с), 33 Мб

Всего запусков: 12, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе решения данной задачи взял на заметку, что формат может заменить алгоритм. Номера вершин упорядочены так, что родитель всегда идет раньше детей, поэтому обход дерева не понадобился - достаточно было сделать один проход по массиву с конца.

Вывод

В рамках выполнения данной работы познакомились с BST, операции с ними, с поворотами.